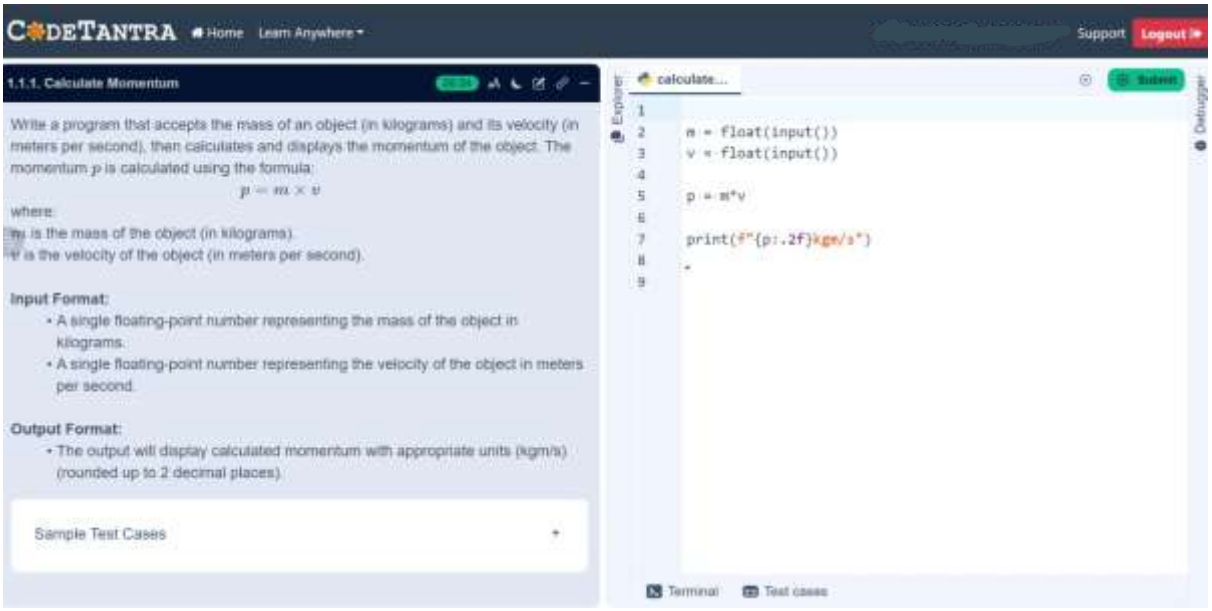
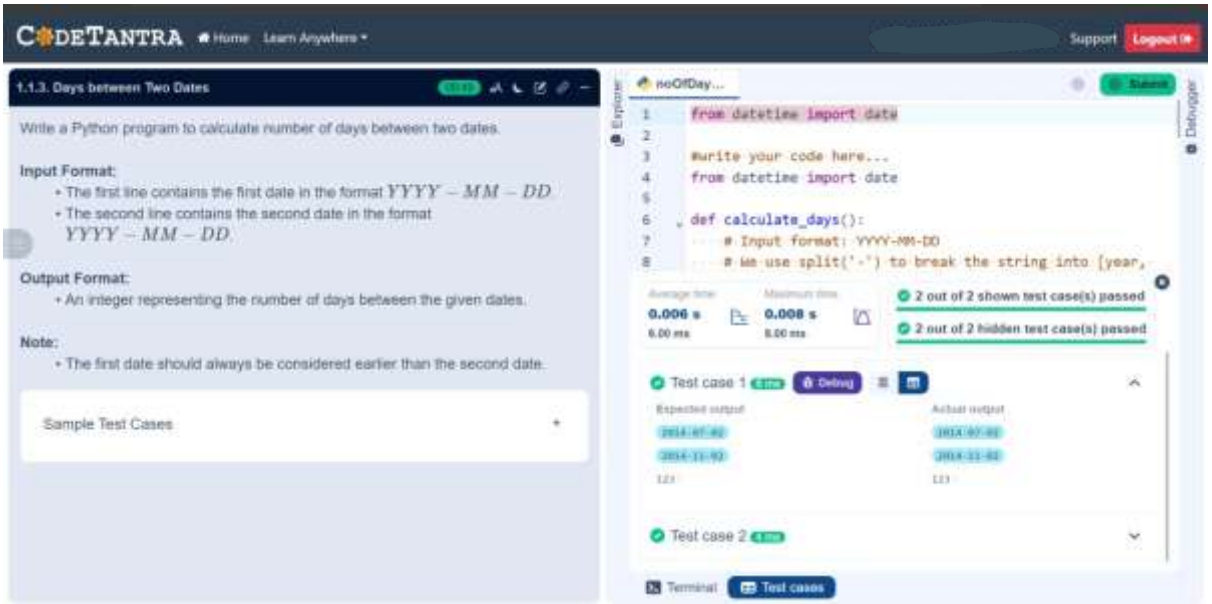
EDS Practice Lab Assignments

Name: Ambika Jakhar

Div: CS3 Batch: Cs-35

PRN: 202501040332





CODETANTRA Home Learn Anywhere Support Logout

1.1.4. Reverse a Number

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Sample Test Cases

reverseN...

```
#write your code here...
# 1. Take input from the user
num = int(input())

reversed_num = 0

# 2. Loop until the number becomes 0
while num > 0:
```

Average time 0.003 s 3.40 ms

Maximum time 0.006 s 6.00 ms

2 out of 2 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 Debug

Expected output

Actual output

Test case 2

Terminal Test cases

CODETANTRA Home Learn Anywhere Support Logout

1.1.5. Multiplication Table

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

```
<number> x <multiplier> = <result>
```

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

Sample Test Cases

multiplica...

```
#write your code here...
# Take an integer input from the user
number = int(input())

# Loop from 1 to 10
for i in range(1, 11):
    # Calculate the result
    result = number * i
```

Average time 0.003 s 3.00 ms

Maximum time 0.004 s 4.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 Debug

Expected output

Actual output

Terminal Test cases

Prev Reset Submit Next

CODETANTRA Home Learn Anywhere Support Logout

2.1.1. List operations

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- **Add**: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove**: Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display**: Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit**: Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Sample Test Cases

listOps.py

```
# write the code..
def main():
    my_list = []

    while True:
        # Display the Menu
        print("1. Add")
        print("2. Remove")
```

Average time 0.035 s 35.00 ms | Maximum time 0.054 s 54.00 ms

2 out of 2 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1 Debug

Expected output	Actual output
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 1	Enter choice: 1
Integer: 11	Integer: 11

Terminal Test cases

Prev Reset Submit Next

CODETANTRA Home Learn Anywhere Support Logout

2.1.2. Dictionary Operations

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

DictOper...

```
# Initial dictionary with 10 predefined records
student = {
    1: "Amit",
    2: "Riya",
    3: "Kiran",
    4: "Neha",
    5: "Arjun",
    6: "Pooja",
```

Average time 0.011 s 10.75 ms | Maximum time 0.013 s 13.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 Debug

Expected output	Actual output
Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}	Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
11	11
Suresh	Suresh
After Insertion: {1: 'Amit', 2: 'Riya',	After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arju

Terminal Test cases

CODETANTRA Home Learn Anywhere Support Logout

3.1.1. Numpy Array Operations

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, r and c, representing the number of rows and the number of columns.
- The next r lines contain c space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use `reshape()` function to reshape the input array with the specified number of rows and columns.

numpyarr...

```
import numpy as np

# write your code here...

# Step 1: Read the number of rows (r) and columns (c)
r, c = map(int, input().split())

# Step 2: Read the elements row by row and store them in
```

Average time 0.007 s 7.00 ms | Maximum time 0.011 s 11.00 ms

2 out of 2 shown test case(s) passed

10 out of 10 hidden test case(s) passed

Test case 1 Debug

Expected output

```
1 4
1 2 3 4
5 6 7 8
9 10 11 12
[[ 1  2  3  4]
 [ 5  6  7  8]
```

Actual output

```
2 4
1 2 3 4
5 6 7 8
9 10 11 12
[[ 1  2  3  4]
 [ 5  6  7  8]
```

Terminal | Test cases

Prev | Reset | Submit | Next

30°C Clear | Search | ENG IN | 11:41 PM 08-05-2026

CODETANTRA Home Learn Anywhere Support Logout

4.1.1. Pandas - series creation and manipulation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases

seriesMa...

```
import pandas as pd

# Take inputs from the user to create a list of numbers
numbers = list(map(int, input().split()))

# Create a Pandas series from the list of numbers
s = pd.Series(numbers)
# Grouping by even and odd numbers and calculating the
```

Average time 0.009 s 9.83 ms | Maximum time 0.017 s 17.00 ms

3 out of 3 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 Debug

Expected output

```
1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers:
Odd    5.0
Even   6.0
dtype: float64
```

Actual output

```
1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers:
Odd    5.0
Even   6.0
dtype: float64
```

Terminal | Test cases

Prev | Reset | Submit | Next

30°C Clear | Search | ENG IN | 11:42 PM 08-05-2026

CODETANTRA Home Learn Anywhere Support Logout

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

datafram...

```python
import pandas as pd

# Provided dictionary of lists
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
}
```

Average time 0.058 s 57.50 ms | Maximum time 0.062 s 62.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1 Debug

Expected output

```
Original DataFrame:
    Name  Age
0  Alice   25
1    Bob   30
2Charlie   35
New name: Susan
```

Actual output

```
Original DataFrame:
    Name  Age
0  Alice   25
1    Bob   30
2Charlie   35
New name: Susan
```

Terminal | Test cases

< Prev | Reset | Submit | Next >

30°C Clear | Search | ENG IN | 11:42 PM 08-05-2026

CODETANTRA Home Learn Anywhere Support Logout

4.1.3. Student Information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is "B").

Note:

Refer to the displayed test cases for better understanding.

Sample Test Cases +

studentin...

```python
import pandas as pd

# Read the text file into a DataFrame
file = input()
data = pd.read_csv(file, sep="\s+", header=None, names=
["Name", "Age", "Grade"])

print("First five rows:")
```

Average time 0.026 s 26.00 ms | Maximum time 0.038 s 38.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1 Debug

Expected output

```
studentdata.txt
First five rows:
   Name  Age Grade
0  John   25     A
1  Alic   22     B
2  Emma   24     A
```

Actual output

```
studentdata.txt
First five rows:
   Name  Age Grade
0  John   25     A
1  Alic   22     B
2  Emma   24     A
```

Terminal | Test cases

< Prev | Reset | Submit | Next >

CODETANTRA Home Learn Anywhere Support Logout

5.1.1. Stacked Plot

Create a stacked area plot to visualize the temperature variations for three different cities (City A, City B, and City C) across the months of the year. The temperature data is provided for each city in the editor.

Your task is to:

- Create a stacked area plot using the data.
- Label the x-axis as "Month", the y-axis as "Temperature", and provide the title "Temperature Variation" for the plot.
- Display the plot showing the temperature variation for each city throughout the months of the year.

Sample Test Cases +

stackedpl... Submit

```
import matplotlib.pyplot as plt
import pandas as pd

# Data for Months and Temperature for three cities
data = {
    'Month': ['January', 'February', 'March', 'April',
'May', 'June', 'July', 'August', 'September', 'October',
'November', 'December'],
```

Average time 0.177 s 177.00 ms

Maximum time 0.177 s 177.00 ms

1 out of 1 shown test case(s) passed

Test case 1 Debug

Expected output

Actual output

Terminal Test cases

Prev Reset Submit Next